UT1 · KOTLIN FUNDAMENTALS

Block 1

Getting started, variables, types and operators

What you will learn
Why we use Kotlin and how to create and run a Kotlin project in IntelliJ IDEA
Write your first program: `main`, `println` and comments
Declare values and variables with `val` and `var`, and understand type inference
Use the basic types, arithmetic, comparison and logical operators
Build text with string templates and convert between types safely
Avoid the most common mistakes with numbers (integer division, precision, overflow)

This block assumes you already know object-oriented programming in Java (UT0). Most ideas are not new; what changes is the **syntax** and some **safer defaults**. Throughout these notes, the orange boxes compare Kotlin with the Java you already know.

Contents

Contenido

Contents....1
1. Why Kotlin?....3
2. Setting up the environment....3
2.1 Creating a Kotlin project in IntelliJ IDEA....3
2.2 Project structure and running the program....3
3. Your first program....4
3.1 print and println....4
3.2 Comments....4
4. Variables: val and var....5
4.1 Type inference....5
5. Basic types....6
6. Operators....7
6.1 Arithmetic operators....7

6.2 Comparison operators 7
6.3 Logical operators 8
7. String templates 8
8. Type conversions 9
8.1 From text to number 9
8.2 Reading input from the keyboard 9
9. Numeric types: common mistakes 10
9.1 Decimal precision 10
9.2 Overflow 10
10. Good practices 11
11. Summary: Java vs Kotlin 11
12. Exercises 13
Exercise 1 · Introduce yourself 13
Exercise 2 · val or var? 13
Exercise 3 · Find the errors 13
Exercise 4 · Temperature converter 14
Exercise 5 · Price, discount and tax calculator 14
Extension · Budget receipt 14

1. Why Kotlin?

Kotlin is a modern programming language created by JetBrains (the company behind IntelliJ IDEA). It runs on the Java Virtual Machine (JVM), so it can use any Java library, and Java and Kotlin code can live in the same project.

Since 2019, Google has declared Android development **Kotlin-first**. Jetpack Compose, the UI toolkit we will use for the rest of the course, is written in Kotlin and can only be used from Kotlin. That is why this unit exists: before building interfaces, we need to be comfortable with the language.

The main advantages of Kotlin for us are:

- **Concise:** the same program usually needs far fewer lines than in Java.
- **Safe:** the compiler prevents many errors before the program runs, especially the famous `NullPointerException`.
- **Interoperable:** it understands Java code, so everything you learned in UT0 is still useful.
- **Expressive:** features such as default parameters and lambdas (coming in later blocks) are exactly what Compose relies on.

2. Setting up the environment

2.1 Creating a Kotlin project in IntelliJ IDEA

IntelliJ IDEA includes Kotlin support out of the box; there is nothing extra to install. To create a project:

1. Open IntelliJ IDEA and choose **File** → **New** → **Project**.
2. In the list on the left, select **Kotlin**.
3. Give the project a name (for example `ut1-block1`) and choose a location.
4. Build system: **Gradle** (with Kotlin DSL) is the one Android uses, so it is a good habit. **IntelliJ** is also fine for simple console programs.
5. Select a JDK (17 or newer) and tick **Add sample code**. Click **Create**.

2.2 Project structure and running the program

IntelliJ creates a file called `Main.kt` inside `src/main/kotlin`. Kotlin source files use the `.kt` extension. To run the program, click the green ▶ icon that appears in the margin next to `fun main()`, or press **Shift + F10** after the first run. The output appears in the **Run** window at the bottom.

> **TIP · Organise your exercises**
> You can have several files in the same project, each with its own `main` function. Create one file per exercise (right-click on `kotlin` → **New** → **Kotlin Class/File** → **File**) and run the one you need with its own ▶ icon.

3. Your first program

Main.kt
```
// My first Kotlin program
fun main() {
    println("Hello, Kotlin!")
}
```

Even this tiny program shows several differences from Java:

- Functions are declared with the keyword `fun`.
- `main` does not need to be inside a class, and it does not need `public static void` or the `String[] args` parameter.
- `println` replaces `System.out.println`.
- **Semicolons are optional.** By convention, we do not write them.

Java
```
public class Main {
    public static void main(String[] args) {
        System.out.println("Hello!");
    }
}
```

Kotlin
```
fun main() {
    println("Hello!")
}
```

3.1 print and println

`println` writes the text and then moves to a new line. `print` writes the text and stays on the same line.

```
fun main() {
    print("Hello, ")
    print("world")
    println("!")
    println("This is a new line")
}
```

Output
```
Hello, world!
This is a new line
```

3.2 Comments

Comments work exactly as in Java. Use them to explain **why** the code does something, not to repeat **what** it obviously does.

```
// Single-line comment

/*
   Multi-line comment
*/

/**
 * Documentation comment (KDoc), the Kotlin equivalent of Javadoc.
 */
```

4. Variables: val and var

Kotlin has two keywords to declare variables:

Keyword	Meaning	Can it change?	Java equivalent
`val`	value (read-only)	No, it is assigned once	`final` variable
`var`	variable	Yes, it can be reassigned	normal variable

```
fun main() {
    val name = "Laura"      // read-only
    var age = 19            // can change

    age = 20                // OK
    // name = "Pablo"       // ERROR: Val cannot be reassigned

    println(name)
    println(age)
}
```

TIP · Prefer val

Use `val` by default and change it to `var` only when you really need to modify the value. Code with fewer changing values is easier to read and has fewer bugs. IntelliJ will even suggest it: if a `var` is never reassigned, it underlines it and proposes changing it to `val`.

4.1 Type inference

In the example above we did not write any types. Kotlin **infers** the type from the value: `"Laura"` is a `String`, so `name` is a `String`; `19` is an `Int`, so `age` is an `Int`. Kotlin is still a **statically typed** language: once the type is decided, it cannot change.

```
var age = 19        // age is an Int
age = "twenty"      // ERROR: Type mismatch: inferred type is String but Int was expected
```

You can also write the type explicitly, after the name and a colon. This is necessary when you declare a variable without giving it a value yet.

```
val city: String = "Villarrobledo"
val price: Double = 19.99

val total: Int      // declared without value: the type is required
total = 100         // assigned once, later
```

Java	Kotlin

`final String city = "Villarrobledo";`
`int age = 19;`
`double price = 19.99;`
	`val city = "Villarrobledo"`
	`var age = 19`
	`val price: Double = 19.99`

COMING FROM JAVA · The type goes after the name

In Java you write `type name`. In Kotlin you write `name: type`. This order appears everywhere in Kotlin: variables, function parameters and properties. You will see it constantly in Compose, for example `text: String` or `modifier: Modifier`.

To see the type Kotlin has inferred, place the cursor on the variable and press **Ctrl + Shift + P** in IntelliJ.

5. Basic types

These are the types you will use most often:

Type	Stores	Example literal
`Int`	Whole numbers (32 bits)	`42`, `-7`, `1_000_000`
`Long`	Large whole numbers (64 bits)	`3_000_000_000L`
`Double`	Decimal numbers (64 bits). The default for decimals	`3.14`, `2.0`
`Float`	Decimal numbers with less precision (32 bits)	`3.14f`
`Boolean`	Logical values	`true`, `false`
`Char`	A single character (single quotes)	`'A'`, `'7'`
`String`	Text (double quotes)	`"Hello"`

Underscores in number literals (`1_000_000`) are ignored by the compiler; they only make the number easier to read.

> **COMING FROM JAVA · There are no primitive types in your code**
> In Java you distinguish `int` from `Integer`. In Kotlin there is only `Int`, and it always behaves like an object: you can call functions on it, such as `42.toString()` or `7.coerceAtLeast(10)`. Internally the compiler still uses fast primitive types when it can, so there is no performance cost.
> Note also that type names start with a capital letter: `Int`, `Double`, `Boolean`.

6. Operators

6.1 Arithmetic operators

Operator	Operation	Example	Result
`+`	Addition	`7 + 2`	`9`
`-`	Subtraction	`7 - 2`	`5`
`*`	Multiplication	`7 * 2`	`14`
`/`	Division	`7 / 2`	`3` (both are Int!)
`/`	Division	`7 / 2.0`	`3.5`
`%`	Remainder (modulo)	`7 % 2`	`1`

> **WATCH OUT · Integer division**
> If both operands are `Int`, the result is an `Int` and the decimal part is **discarded** (not rounded). `7 / 2` is `3`, and `1 / 2` is `0`. If you need decimals, at least one operand must be a `Double`: `7 / 2.0` or `7.toDouble() / 2`.

As in Java, there are compound assignment operators (`+=`, `-=`, `*=`, `/=`, `%=`) and the increment and decrement operators (`++`, `--`). They only work with `var`, since they modify the variable.

```
var points = 10
points += 5     // 15
points *= 2     // 30
points++        // 31
println(points)
```

6.2 Comparison operators

`<`, `>`, `<=`, `>=`, `==` and `!=` work as you expect and return a `Boolean`.

> **COMING FROM JAVA · == compares content, also for Strings**
> In Java you must compare strings with `equals()`, because `==` compares references. In Kotlin, `==` calls `equals()` for you, so `name == "Laura"` is the correct way to compare text. If you ever need to check whether two variables point to the same object, Kotlin uses `===`.

6.3 Logical operators

Operator	Meaning	Example (age = 20)	Result
`&&`	AND: true if both are true	`age >= 18 && age < 65`	`true`
`||`	OR: true if at least one is true	`age < 12 || age > 65`	`false`
`!`	NOT: inverts the value	`!(age >= 18)`	`false`

```
val age = 20
val hasTicket = true
val canEnter = age >= 18 && hasTicket
println(canEnter)   // true
```

7. String templates

A **string template** lets you insert values directly inside a text, instead of joining pieces with `+`. Write `$` followed by the variable name. If you need an expression (a calculation, a function call, a property), wrap it in `${ }`.

```
val product = "Keyboard"
val price = 25.0
val quantity = 3

println("Product: $product")
println("Total: ${price * quantity} euros")
println("The name has ${product.length} letters")
```

Output

```
Product: Keyboard
Total: 75.0 euros
The name has 8 letters
```

Java

```
System.out.println("Product: " + product
    + ", total: " + (price * quantity));
```

Kotlin

```
println("Product: $product, total: ${price *
quantity}")
```

> **WATCH OUT · $name vs ${...}**
> `"$product.length"` does **not** print the length: Kotlin inserts `$product` and then writes the text `.length`. Whenever there is anything more than a plain variable name, use braces: `"${product.length}"`.

String templates are used constantly in Compose, for example `Text("Hello, $name!")`.

8. Type conversions

Kotlin **never converts numbers automatically**, not even from a smaller type to a bigger one. This avoids hidden errors, but it means you must convert explicitly using the `toX()` functions.

```
val units: Int = 10
// val big: Long = units         // ERROR: Type mismatch
val big: Long = units.toLong()   // OK

val price = 9.99
val rounded = price.toInt()      // 9 (decimals are discarded)
val asText = price.toString()    // "9.99"
```

> **COMING FROM JAVA · No implicit widening**
> In Java, `long big = units;` compiles because Java widens `int` to `long` automatically. In Kotlin it does not compile: you must write `units.toLong()`.

8.1 From text to number

Text can be converted into numbers with the same functions. If the text is not a valid number, the program stops with a `NumberFormatException`. The `...OrNull` versions return `null` instead of failing; we will learn to work with `null` safely in Block 4.

```
val a = "42".toInt()             // 42
val b = "3.5".toDouble()         // 3.5
// val c = "abc".toInt()         // crashes: NumberFormatException
val d = "abc".toIntOrNull()      // null
```

8.2 Reading input from the keyboard

`readln()` reads a line typed by the user and returns it as a `String`. To get a number, convert the result.

```
fun main() {
    print("What is your name? ")
    val name = readln()

    print("How many units? ")
    val quantity = readln().toInt()

    println("$name wants $quantity units")
}
```

9. Numeric types: common mistakes

9.1 Decimal precision

`Double` and `Float` store numbers in binary, and many decimal values cannot be represented exactly. Small errors appear in the last digits:

```
println(0.1 + 0.2)          // 0.30000000000000004
println(0.1 + 0.2 == 0.3)   // false
```

This is not a Kotlin bug: Java and almost every other language behave the same way. In practice it means two things:

- **Format** decimal results before showing them to the user.
- Do not compare decimal numbers with `==` when exactness matters.

To show a number with two decimals, use `format`:

```
import kotlin.math.roundToInt   // imports go at the top of the file

fun main() {
    val total = 0.1 + 0.2
    println("%.2f".format(total))   // 0,30 or 0.30 (depends on the system language)
    println(2.7.roundToInt())       // 3 (rounds to the nearest Int)
}
```

> **TIP · Money in real applications**
> Professional apps that handle money do not use `Double`; they use `BigDecimal` or store amounts in cents as `Long`. For the exercises in this unit, `Double` with formatted output is enough.

9.2 Overflow

An `Int` can only store values between about −2,100 million and +2,100 million. If a calculation goes beyond the limit, the value **wraps around** without any warning:

```
val max = Int.MAX_VALUE      // 2147483647
println(max + 1)             // -2147483648
val safe = max.toLong() + 1  // 2147483648
```

If your values can be very large (populations, milliseconds, file sizes), use `Long`.

10. Good practices

- **Descriptive names in camelCase:** `unitPrice`, `discountPercentage`. Avoid names such as `x`, `aux` or `p2`.
- **Use val by default**, and `var` only when the value changes.
- **Explicit types when they help the reader.** `val total = calculate()` hides the type; `val total: Double = calculate()` makes it clear. For simple literals (`val age = 19`) inference is enough.
- **Constants:** values that never change and are known before the program runs are declared at the top of the file, outside any function, with `const val` and in UPPER_SNAKE_CASE.

```
const val DEFAULT_TAX = 21.0
const val SHOP_NAME = "DAM Store"

fun main() {
    println("Welcome to $SHOP_NAME")
    println("Tax applied: $DEFAULT_TAX%")
}
```

11. Summary: Java vs Kotlin

Concept	Java	Kotlin
Entry point	`public static void main(String[] args)`	`fun main()`
Print a line	`System.out.println(x);`	`println(x)`
Semicolons	Required	Optional (not used)
Constant variable	`final int x = 5;`	`val x = 5`
Mutable variable	`int x = 5;`	`var x = 5`
Type position	`String name`	`name: String`
Primitive types	`int`, `double`, `boolean`	`Int`, `Double`, `Boolean`
Compare text	`a.equals(b)`	`a == b`
Join text	`"Hi " + name`	`"Hi $name"`
int → long	Automatic	`x.toLong()`

Concept	Java	Kotlin
Read a line	`new Scanner(System.in).nextLine()`	`readln()`

12. Exercises

Create one Kotlin file per exercise inside your `ut1-block1` project. Use `val` unless you need `var`, and use string templates for every output.

Exercise 1 · Introduce yourself

Declare values for your name, age, city and whether you have a driving licence (`Boolean`). Print a sentence like this using a single string template:

> **Expected output**
> `My name is Laura, I am 19 years old, I live in Villarrobledo. Driving licence: true`

Then add a line that shows how old you will be in 10 years, calculated inside the template.

Exercise 2 · val or var?

For each piece of data, decide whether you would declare it with `val` or `var`, choose its type, and justify your answer in a comment:

1. Your date of birth (year).
2. The number of lives left in a game.
3. The name of the app.
4. The items currently in a shopping cart (just the counter).
5. The value of π.

Exercise 3 · Find the errors

Each of the following fragments has a problem: some do not compile and others compile but give an unexpected result. Explain each problem and fix it.

```
// a)
val score = 10
score = score + 5

// b)
var temperature = 21
temperature = 21.5

// c)
val units: Int = 3
val total: Long = units

// d)
val average = 7 / 2
println("Average: $average")   // we expected 3.5

// e)
val word = "Kotlin"
println("Letters: $word.length")
```

Exercise 4 · Temperature converter

Ask the user for a temperature in degrees Celsius with `readln()`, convert it to Fahrenheit using the formula `F = C × 9 / 5 + 32`, and show the result with one decimal.

> **Example**
>
> ```
> Temperature in Celsius: 21.5
> 21.5 °C = 70.7 °F
> ```

> **TIP · Think about it**
>
> What happens if you write the formula as `c * (9 / 5) + 32`? Try it and explain the result.

Exercise 5 · Price, discount and tax calculator

Create a calculator that reads the following data from the keyboard:

- Product name
- Unit price
- Quantity
- Discount (%)
- Tax (%)

The program must calculate and display:

Value	Example: unit price 25, quantity 4, discount 10 %, tax 10 %
Subtotal	100.00
Discount amount	10.00
Total price after discount	90.00
Tax amount	9.00
Total	99.00

Requirements: use descriptive names, `val` for every value that does not change, and show all amounts with two decimals.

Extension · Budget receipt

Improve Exercise 5 so that the result is printed as a receipt:

- Store the shop name and the default tax in `const val` constants. If the user presses Enter without typing a tax, use the default value. (Hint: `readln().toDoubleOrNull() ?: DEFAULT_TAX`. We will explain the `?:` operator in Block 4.)
- Align the amounts in a column. Hint: `"%10.2f".format(total)` reserves 10 characters for the number.

Example

```
========== DAM Store ==========
Product:   Keyboard x 4
Subtotal:              100.00
Discount (10.0%):      -10.00
After discount:         90.00
Tax (10.0%):             9.00
------------------------------
TOTAL:                  99.00
```